

Atlas Marketplace

A concept and technical approach for Greenstone's Senior Frontend Engineer interview task

Prepared by Ravi Kumar · 5 October 2026

Executive summary

Atlas Marketplace is a **private-markets placement marketplace**. Fund managers (GPs) list fund vehicles seeking capital; investors (LPs — family offices, sovereign wealth funds, institutions, UHNWIs) discover, diligence, submit indications of interest, get allocated, and close. It deliberately mirrors the category of problem Greenstone's own placement business solves manually today — GCC investors connecting with global fund managers — without describing Greenstone's existing platform. That choice is the foundation for everything that follows: the data model is a capital-markets book-building process, not an e-commerce cart; the "bidding" mechanic is an allocation decision under real regulatory and relationship constraints, not an auction.

A working prototype accompanies this document, built in React, TypeScript and MUI on the stack Greenstone's brief specifies: atlas-marketplace-sigma.vercel.app.

The brief: category and participants

What's traded. Access to private-market fund vehicles — commitments of capital into closed-end funds (growth equity, venture, private credit, real assets) managed by professional fund managers.

Why this category suits a marketplace model. Capital placement today is relationship-driven and largely offline — exactly the high-friction, information-asymmetric market a marketplace model disintermediates. It has a genuine two-sided cold-start problem (GPs need credible LPs; LPs need vetted GPs), real listing/valuation questions (track record, fees and terms, not a fixed price), and a real negotiation dynamic (funds get oversubscribed and must allocate). It is not a generic category that would fit any marketplace template — which is the point: it is a stronger demonstration of product thinking than a physical-goods or collectables marketplace would be, and one that speaks directly to the domain this panel works in every day.

Sellers — Fund Managers (GPs). Need credible, qualified demand for a raise; a way to show track record and differentiate from other funds currently raising; a controlled, auditable process for allocating an oversubscribed fund; compliant handling of sensitive LP data during diligence.

Buyers — Investors (LPs). Family offices, sovereign wealth funds, institutions and UHNWIs. Need a trustworthy way to discover funds that fit their mandate (strategy, geography, ticket size, vintage); enough diligence material to compare funds without a GP's IR team as the only channel; a clear, provable path from expressing interest to a signed subscription; confidence the platform enforces accreditation on every counterparty, not just claims to.

Required marketplace area 1 — Buyer and seller relationship

Every organization on the platform is either a `FUND_MANAGER` or an `INVESTOR`, with an explicit `accreditation` status (`VERIFIED` / `PENDING` / `UNVERIFIED`) that gates what they can do — an unverified LP can browse but not message a GP or submit an indication. GPs get a firm-level profile carrying their prior funds' track record (`TrackRecordEntry` , deliberately modeled on the organization, not a single fund vehicle, because a firm's history is evidence for every current raise). LPs get a profile carrying their investor tier and stated ticket-size range, which drives discovery matching.

Discovery runs both directions: LPs filter fund vehicles by strategy, geography, vintage and fit against their own ticket range; GPs see aggregated demand signals without per-fund investor lists being casually browsable. Trust is established through Greenstone verifying every organization before it can transact — not an algorithmic trust score in v1, a human review step, because that is cheaper and more credible for this asset class than trying to solve cold-start trust computationally. Communication happens in per-fund message threads, gated behind verified accreditation, so diligence Q&A has a permanent, auditable record instead of living in email.

Required marketplace area 2 — Listing and valuation

A GP creates a `FundVehicle` listing in `DRAFT` and cannot publish it until track-record documents and a data room are attached — the platform will not let an under-prepared raise go live. A published listing carries strategy, vintage, geography, target size, hard cap, minimum commitment, management fee and carry, a rich narrative (authored and rendered through Tiptap), and a benchmark performance chart (fund IRR plotted against a strategy benchmark, built with Recharts). There is no single "price" to discover — value is established through structured, comparable fields (IRR, MOIC, fees) and supporting documents in the data room, which LPs can request more of through the message thread. A buyer can compare listings on these structured terms directly, not just read prose and guess.

Required marketplace area 3 — Bidding and purchase

An LP submits an **Indication of Interest** — a requested commitment amount, not a fixed bid — against a fund vehicle. What happens next is the core technical problem this brief asks about, and it is answered concretely, not abstractly: see the state diagram and the "Real-time and transaction

states" section below for the full lifecycle and the exact mechanism that prevents two LPs racing near a fund's hard cap from producing an inconsistent outcome. In short: allocation is **server-authoritative** (decided against the fund's live committed total at the moment of submission, not whatever the client last rendered), submissions are **idempotent** (a dropped network response and its retry can never create a duplicate indication), and oversubscription is resolved by **pro-rata scale-back** with a GP override available for a documented reason — because real fundraising legitimately prioritizes strategic LPs, and pretending otherwise would be naive, not rigorous.

Principal user journeys

GP journey — raise a fund. Register the firm → Greenstone verifies accreditation and compliance → create a `FundVehicle` listing → publish (`OPEN`) → monitor live subscription progress and incoming indications → review submitted indications, declining with a required reason where necessary → as the fund approaches its hard cap, allocations resolve automatically with pro-rata scale-back → send subscription documents → track KYC/AML and signature completion → close the fund (`FUNDED`).

LP journey — commit capital. Register the organization → complete an accreditation profile → browse and filter fund vehicles → open a fund's detail page: narrative, track record chart, data room → message the GP with diligence questions → submit an Indication of Interest → track status live through allocation, acceptance, subscription documents and signature → fund the commitment.

Key decision points. Whether to publish before track-record documents are ready (blocked by design). What happens when total demand exceeds the hard cap (pro-rata, not first-come-first-served). Whether an LP can withdraw before allocation completes (yes). What happens if the close date passes under target (a GP decision, not an automated rule — real fundraises are extended or closed by human judgement, not a cron job).

Marketplace mechanics

- **Discovery & search.** Filter by strategy, geography, vintage, and fit against the LP's own stated ticket range.
- **Comparison.** Structured, filterable fields (IRR, MOIC, fees) plus a benchmark performance chart on every listing — comparison by data, not by sales narrative alone.
- **Communication.** Per-fund message threads, gated behind verified accreditation.
- **Trust & safety.** Organization-level verification, accreditation status visible on every profile, a full audit log on every indication state change.

- **Payment & fulfilment.** Deliberately out of scope for in-platform money movement in v1 — the platform tracks the commitment and subscription lifecycle; actual capital calls and wiring run through the fund's existing administrator. Building escrow and payment rails into a v1 would be a far larger regulatory and engineering commitment than this release should carry, for a workflow that already has an established, working channel.
 - **Disputes.** A declined or scaled-back indication carries a required GP-authored reason, visible to the LP and preserved in the audit log for Compliance review.
 - **Regulation.** DIFC data-protection framing runs through the design, not bolted on at the end: accreditation/KYC status is a first-class field on every organization, and a dedicated COMPLIANCE role has read/audit access with no allocation authority — deal-making and oversight are structurally separated.
-
-

Priorities for an initial release

In v1: listing creation and publish flow, discovery and filtering, structured comparison, messaging, indication submission with server-authoritative allocation and pro-rata scale-back, live subscription-progress updates, tracked (not generated) subscription-document handoff, and role-based access across LP/GP/Admin/Compliance.

Explicitly deferred: in-platform capital movement and escrow (regulatory and integration scope far beyond a v1, and funds already have administrators for this); automated KYC/AML provider integration (mocked as a status field — vendor selection is a compliance and procurement decision, not a frontend architecture one); secondary transfers of existing LP stakes (a related but materially different product, considered and deliberately set aside); multi-currency support (USD-denominated commitments cover the large majority of GCC institutional tickets).

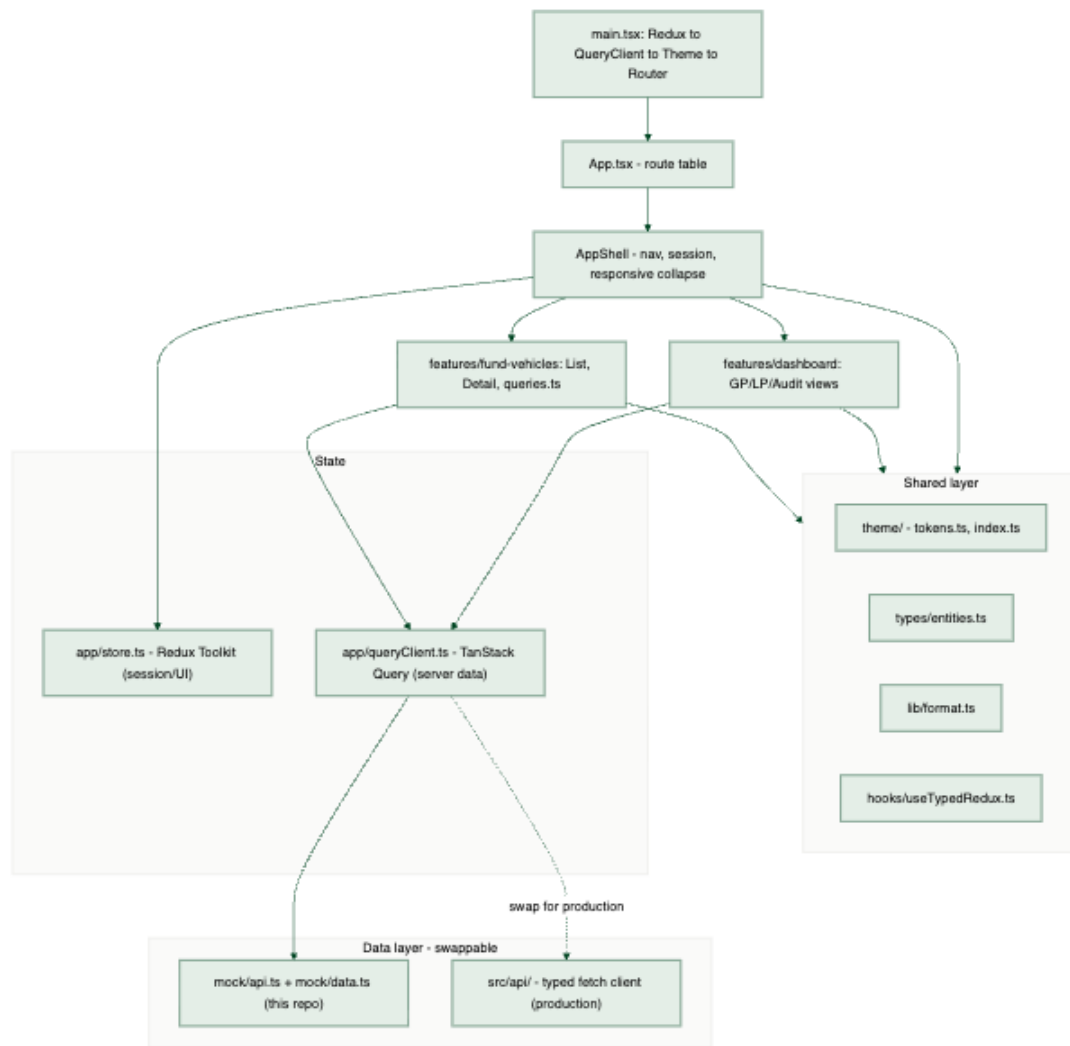
Assumptions. GPs and their fund vehicles are pre-vetted by Greenstone before they can publish — cold-start trust is solved by a human review step, not an algorithm, because that is more credible for this asset class. Allocation defaults to pro-rata but a GP can override with a logged reason, because pure algorithmic allocation would misrepresent how real fundraising actually works.

Technical approach

Frontend architecture

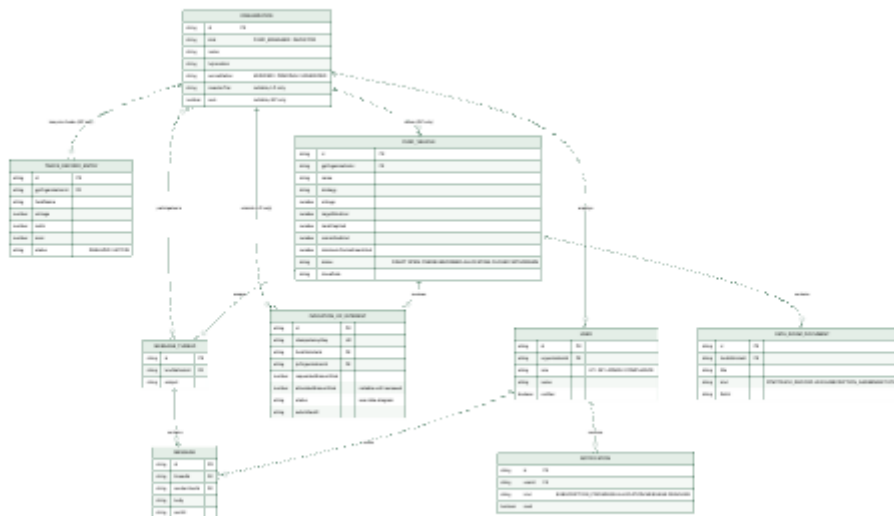
React, TypeScript, Vite and MUI, themed to Greenstone's actual brand — colors and typography pulled live from gsequity.com, not guessed, and documented with measured WCAG contrast ratios in the repository's DESIGN.md. Routing via React Router. Client/session state lives in Redux Toolkit; all server data lives in TanStack Query — a deliberate split, not two libraries doing the same job: Redux never caches anything the API returned, and swapping the mock API for a real one touches zero Redux code.

Vite was chosen over Next.js for this build: the JD lists both across Greenstone's products, and Atlas Marketplace is an authenticated, behind-login application where server-side rendering's main benefit — first-paint SEO — doesn't apply to most of the surface. A public, unauthenticated discovery page would be the concrete trigger to introduce Next.js for that slice specifically.



Data model and API

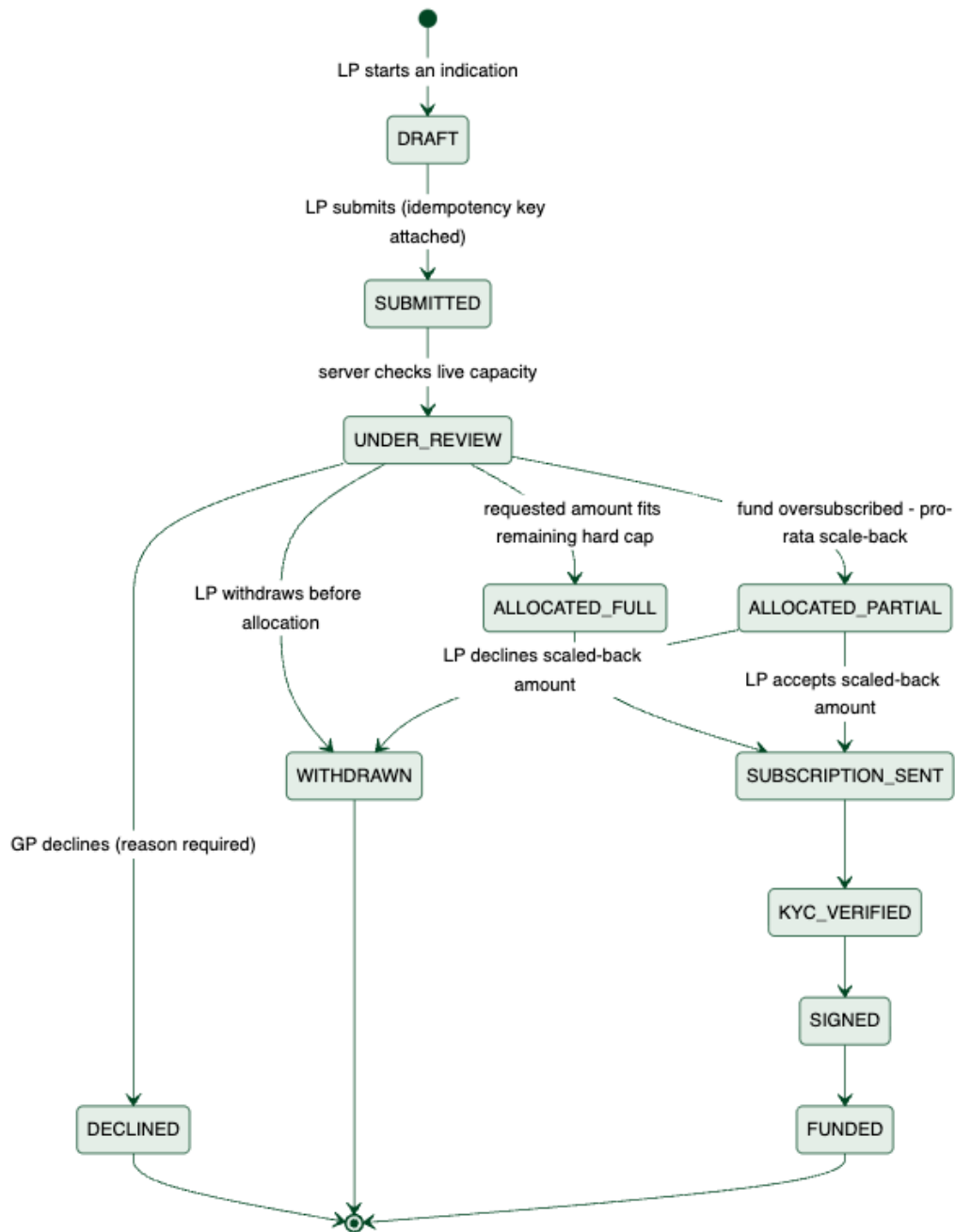
Core entities: `User`, `Organization`, `FundVehicle`, `TrackRecordEntry`, `IndicationOfInterest`, `MessageThread` / `Message`, `DataRoomDocument`, `Notification` — full relationships in the entity-relationship diagram. The REST surface (Laravel, versioned under `/api/v1`) includes fund-vehicle CRUD and publish, an indications endpoint with server-computed allocation, organization profiles, signed data-room URLs, and per-fund messaging. The frontend consumes all of it through typed TanStack Query hooks, one `queries.ts` per feature area.



Real-time and transaction states

The full Indication-of-Interest lifecycle — from `DRAFT` through `SUBMITTED`, `UNDER_REVIEW`, `ALLOCATED_FULL` / `ALLOCATED_PARTIAL`, `SUBSCRIPTION_SENT`, `KYC_VERIFIED`, `SIGNED`, to `FUNDED`, with `DECLINED` and `WITHDRAWN` off-ramps — is implemented as an explicit, enforced state machine: an illegal transition (for example, withdrawing an indication that has already been signed) is rejected outright, not silently allowed. This is directly demonstrated in the prototype and covered by unit tests, not just described.

Concurrency. Two LPs racing to submit indications near a fund's hard cap both receive an outcome computed against the fund's live, current committed total — never against stale client state. Submissions carry a client-generated idempotency key, so a retried request after a dropped response cannot create a duplicate. In production this capacity check runs inside a single database transaction (row-level lock or optimistic version column with retry) so two near-simultaneous submissions serialize correctly; the prototype's mock API is single-threaded JavaScript and cannot reproduce that race condition itself, so this is documented as the production mechanism, while the scale-back policy it protects is real, working code in the prototype. Live "% subscribed" and allocation notifications are delivered over Pusher private channels in production, authenticated per fund vehicle.



Security

Entra ID SSO for authentication (the JD names Microsoft 365/Entra ID explicitly), role-based access control enforced server-side across LP / GP / ADMIN / COMPLIANCE — never only hidden in the UI. Sensitive financial and KYC data is encrypted at rest, data-room documents are served through signed, time-limited URLs rather than public links, and every indication state transition is

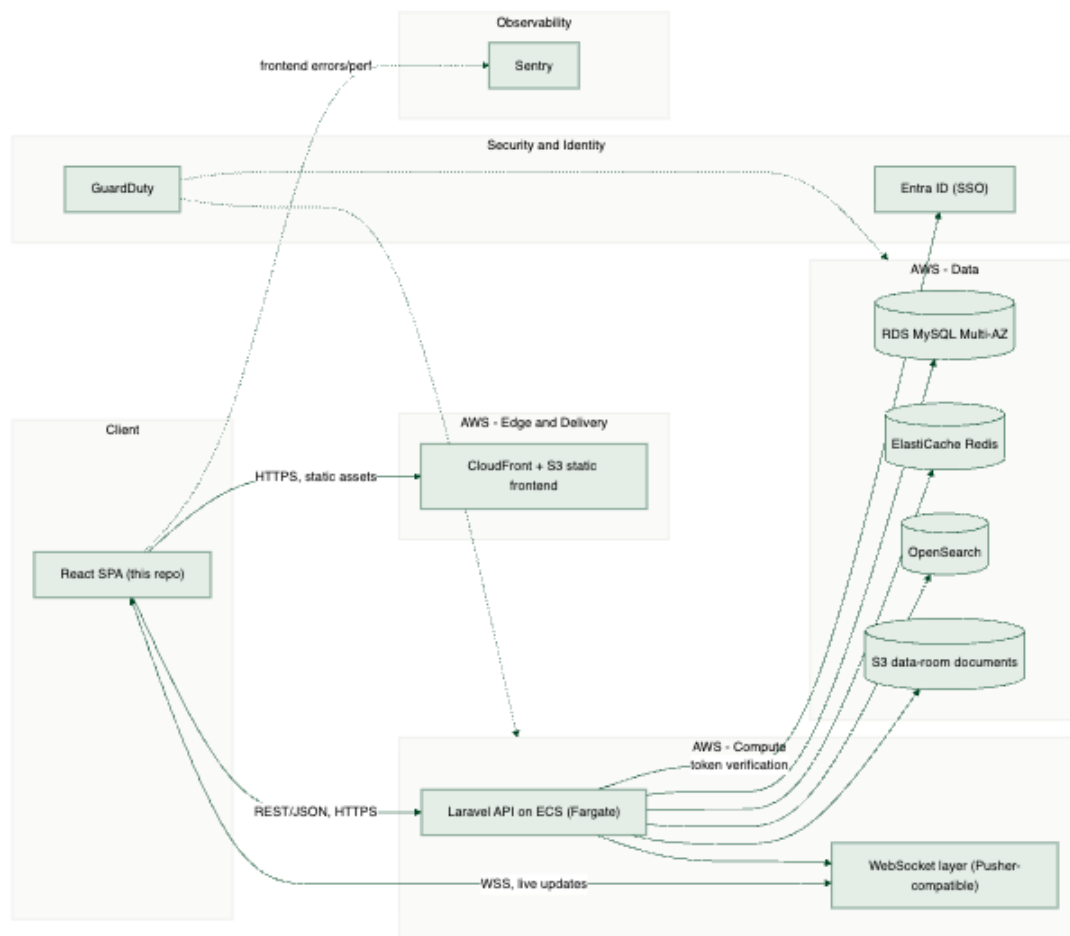
audit-logged for Compliance review and dispute resolution. OWASP Top 10 practices apply throughout: CSRF protection, server-side sanitization of rich-text listing content against stored XSS, and rate limiting on the indication-submission endpoint to prevent scripted manipulation of an oversubscribed allocation. DIFC data-protection requirements shape the data model from the start, not as a later compliance pass.

Performance and quality

Route-level code splitting, MUI X Data Grid's built-in virtualization for the data-dense GP pipeline and LP watchlist views, Sentry for frontend error and performance monitoring, and WCAG AA as the accessibility floor. Testing effort is deliberately concentrated where it matters most — the state-machine and allocation logic, the part of this system most likely to hide a real bug — with component tests for genuinely conditional UI and one end-to-end test over the critical submission path, rather than chasing a coverage percentage across the whole surface.

Delivery

Production: static frontend on S3 and CloudFront, the Laravel API on ECS Fargate behind an ALB, RDS MySQL Multi-AZ, ElastiCache for sessions and real-time presence, OpenSearch for fund-vehicle search once listing volume outgrows client-side filtering, and GuardDuty across the account. CI/CD through GitHub Actions: lint, typecheck and unit tests on every pull request, a preview deployment per PR, promotion to staging on merge, and a manual approval gate to production. Three environments on separate AWS accounts, given the sensitivity of what the platform stores.



Trade-offs

Decision	Alternative considered	Why this way
Redux + TanStack Query split	One library for both	Keeps server-state caching semantics separate from genuinely client-only state; matches the JD's explicit pairing
Vite SPA	Next.js SSR	No public SEO surface for the authenticated core product
Client-side fund filtering	OpenSearch from day one	v1 listing volume doesn't need it; ships faster, upgrades cleanly
Mocked KYC/AML status	Real vendor integration	Vendor selection is a compliance decision, not a frontend one

Decision	Alternative considered	Why this way
Pro-rata scale-back with GP override	Pure priority queue	Matches how real fundraises are actually allocated
Commitment tracking only, no escrow	Full payment processing on-platform	Regulatory scope far exceeds a v1; funds already have administrators

Visual concept

The coded prototype at atlas-marketplace-sigma.vercel.app is the visual concept — higher fidelity than a static wireframe, and a genuine demonstration of the frontend architecture described above rather than a description of it. It covers: the fund-vehicle browse and filter view, a fund detail page with narrative, performance chart, data room and a live Indication-of-Interest form, and role-based dashboards for GP, LP, and read-only Admin/Compliance oversight — all responsive from mobile through desktop.

Supporting links

- Prototype: <https://atlas-marketplace-sigma.vercel.app>
- Source repository and full documentation (`DESIGN.md` , `proposal/` , diagrams, tests): available on request / attached.

AI and tools disclosure

I built this with Claude Code, used the way the JD itself describes using AI tools day to day: as an execution partner I direct, not a black box I accept output from.

What I decided. The category (a private-markets placement marketplace, chosen deliberately to mirror Greenstone's own business) and the scope (a real coded prototype, not just static wireframes, given the JD names Claude Code as an expected tool). The product decisions throughout — required-area coverage, what ships in v1 versus what's deferred and why, the state-machine design for allocation, the trade-offs called out explicitly in the technical approach. Every one of those was a decision point I reviewed and approved before implementation continued, not something generated and left unread.

What Claude Code produced. The scaffold (Vite/React/TypeScript/MUI), the mock data model and API layer, the UI components, the diagrams, and the first drafts of the written material — under my direction and within the architecture I'd approved.

How I checked it. Every phase ran through automated verification before moving on — TypeScript compiling clean, the Vitest suite passing (including the tests for the allocation and state-machine logic, since that's the part of this system most likely to hide a real bug), and the prototype exercised in a real browser at both mobile and desktop widths, not just assumed to work. Real bugs surfaced this way and got fixed as part of the process rather than shipped: a broken mobile layout in the first pass, a TanStack Query gap that turned a missing fund into a fake error page, a Vercel routing gap that 404'd on direct links, and a PDF rendering issue that silently dropped diagram labels. I read this document and the codebase end to end before sending it — including the trade-offs table above, which I'd stand behind in the room regardless of what wrote the first draft of the sentence.